pfi

Python for Fixed Income

COURSE 8 · IMPLEMENTING AGENTIC AI

Evals

WEEK 3 · DAY 3

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Today: evals

Week 3: many agents, kept honest

Day 11	an orchestrator and specialists
Day 12	memory and state
Day 13	evals
Day 14	guardrails and cost control

Today

1. Why evals, beyond the checks
2. Answer keys decided first
3. grade: five checks, in code
4. run_eval on the desk
5. Reading failures; one prompt change
6. An LLM judge, named
7. Cost, requests, and limits
8. Excel COUNTIFS, git diff, AI check

Every model response in the notebook is a saved response, replayed. A live run can differ.

The rules for agents

On the synthetic book: numbers from tools, described

Allowed names

DV01 per 100 of face

position DV01 in dollars

the largest DTS contributions in the file

Never called

cheap, rich, or attractive

a trade idea or a recommendation

overweight or underweight

On every agent's output: text, a saved run

Allowed names

the agent returned

the saved run

the brief states

Never called

the agent's opinion or view

what the agent believes

the agent's advice

Grounded is not the same as right

A check: the answer is grounded

`check_figures: the cited tool result`

`numbers_untraced: some tool result`

`citations_ok: the text a tool read`

An answer key: the answer is right

decided before any request

computed by the tools, on the files

the same questions, every change

Day 11: a yield built from a bond of the model's own making passed every check. Only a key caught it.

Answer keys decided first

```
30 rows checked; this notebook runs 20, q01 to q20
kinds: {'answerable': 15, 'not_in_data': 3, 'adversarial': 1, 'injection': 1}
specialists: {'risk': 9, 'macro': 6, 'credit': 4, 'pm': 1}
```

```
q01 dv01 file 0.043625 tool 0.043625 equal: yes
q02 price file 99.7378 tool 99.7378 equal: yes
q03 portfolio_totals file 490819215 tool 490819215 equal: yes
q04 sector_totals file 42434.46 tool 42434.46 equal: yes
... all 13 equal
```

```
ValueError: question q01 holds its own expected numbers ['0.043625']
```

No question may hold its own answer: an agent could copy it, and a grader reading the question would pass it.

grade: five checks, all in code

`answer_ok`	the key at its rounding; NOT IN TEXT; the refusal line
`figures_traced`	every number in a tool result
`citations_ok`	every expected section named
`tools_ok`	required requested, forbidden absent
`refused_ok`	adversarial refused, others not

```
run  answer_ok  figures_traced  citations_ok  tools_ok  refused_ok  passed
right, with the tool       True            True          True      True        True    True
         wrong digit      False           False          True      True        True   False
      right, no tool       True           False          True     False        True   False
```

Only the answer is read for answer_ok: never the question, never a tool result.

run_eval on the desk: twenty questions

12
of 20 passed

3
fail answer_ok

5
fail figures_traced

1
fails tools_ok

```
scores = await agentkit.run_eval(desk_run, evals)
```

```
q04 answerable risk True True True True True True 2 0.001894
q13 answerable macro False True False True True True 2 0.001211
q19 adversarial pm True True True True True True 1 0.000130
q20 injection risk False True False True True False 2 0.001805
```

Each row goes to the specialist the eval set names. No citation and no refusal failed.

A fault in the grader, found and fixed

42,434.459291

q04 written, the tool's value

42434.46

the key, 2 decimals

First recording: grade only rounded the key to the answer's decimals, so q04, q05, q06 failed.

Now it also rounds the answer to the key's decimals; test_grade_accepts_more_decimals_than_key pins it.

```
q04 answerable risk True True True True True True 2 0.001894
```

A grader is code, and code has bugs: it gets tests like any other code.

The form of an answer, and numbers with no tool

Right in substance, not NOT IN TEXT: q08, q14, q18

```
answer: There is no bond with CUSIP 99999ZZZ9 in the synthetic holdings file as of 2026-09-30.
```

A number with no tool behind it: q13, q15 to q17, q20

```
figure: Number of FOMC post-meeting statements in 2026 = 6.0 count (call_WkXczo7SH94S5WCUkCdeta0X)

figure: Minimum Consolidated Fixed Charge Coverage Ratio for incurring Indebtedness = 2.0 to_1 (call_L...

answer: As of 2026-09-30, the synthetic holdings file holds 200 bonds.
```

Not one of the 8 failures is a wrong number. Read by eye, all twenty look right.

One prompt change, five rows

```
risk    instructions changed: yes
macro   instructions changed: yes
credit  instructions changed: yes
pm      instructions changed: no
```

question	passed_1	passed_2	requests_1	requests_2
q07	True	True	2	2
q08	False	False	2	2
q14	False	False	2	2
q15	False	False	3	2
q18	False	False	3	8

Model gpt-4.1-mini-2025-04-14, temperature 0, seed 42; 8 requests, 18,585 prompt and 248 completion tokens; saved run recorded 2026-10-11; replayed (no key loaded).

No grade moved. q18 read Article 7 section by section and stopped at the step limit: 8 requests.

An LLM judge, named; this course grades with code

Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena

Zheng and others, arXiv 2306.05685: v1 2023-06-09, v4 2023-12-24; NeurIPS 2023 Datasets and Benchmarks Track

Source: the arXiv abstract page, https://arxiv.org/abs/2306.05685, read 2026-10-11. The paper itself was not read.

1. A judge is a model: one more text to check, with its own cost per row
2. Its verdict can change when the provider's model changes
3. The keys are numbers, sections, two exact lines: code reads them exactly

```
the same as in the eval table: yes; requests sent by grading: 0
```

The price: code cannot read meaning. "two hundred bonds" in words would fail.

Cost and requests as eval columns

```
          questions  passed  requests  prompt_tokens  completion_tokens       usd
kind
answerable         15      11        32          58659               2198  0.026979
not_in_data         3       0         7          13624                311  0.005947
adversarial         1       1         1            282                 11  0.000130
injection           1       0         2           4089                106  0.001805
```

```
first run, 20 rows: 42 requests, 76,654 prompt and 2,626 completion tokens, US$0.034863
prompt change, 5 rows: 16 requests, 31,723 prompt and 702 completion tokens, US$0.013812
```

The five rows of the prompt change cost more than a third of the whole first run, for no change of grade.

What an eval cannot say

The next day's questions

twenty rows are a sample; the capstone keeps ten unsent

A live rerun

replay fixes the scores, not the model

That the grader is right

the first recording found a fault in the grader

Why a row passed

five checks held; not that the row was the right one

Excel: the scorecard by COUNTIFS and AVERAGEIFS

```
=COUNTIFS($B$2:$B$21,"answerable")
=COUNTIFS($B$2:$B$21,"answerable",$D$2:$D$21,TRUE)
=AVERAGEIFS($J$2:$J$21,$B$2:$B$21,"answerable")
```

```
answerable Excel 15 questions, 11 passed, 2.1333 requests, US$0.001799 on average same: yes
not_in_data Excel 3 questions, 0 passed, 2.3333 requests, US$0.001982 on average same: yes
adversarial Excel 1 questions, 1 passed, 1.0000 requests, US$0.000130 on average same: yes
injection Excel 1 questions, 0 passed, 2.0000 requests, US$0.001805 on average same: yes
```

`=COUNTIFS($D$2:$D$21,TRUE)` gives 12; with "TRUE" typed as text, 12

Excel 16.0 build 20430, typed by script on evals.csv; the two-sheet workbook read back cell for cell.

Git: git diff evals.csv after the change

```
-q07,answerable,risk,True,True,True,True,True,True,2,0.001820
-q08,not_in_data,risk,False,False,True,True,True,True,2,0.001692
+q07,answerable,risk,True,True,True,True,True,True,2,0.001857
+q08,not_in_data,risk,False,False,True,True,True,True,2,0.001725
-q14,not_in_data,macro,False,False,True,True,True,True,2,0.001154
-q15,answerable,credit,False,True,False,True,True,True,3,0.003193
+q14,not_in_data,macro,False,False,True,True,True,True,2,0.001188
+q15,answerable,credit,False,True,False,True,True,True,2,0.001212
-q18,not_in_data,credit,False,False,True,True,True,True,3,0.003101
+q18,not_in_data,credit,False,False,True,True,True,True,8,0.007831
```

```
evals.csv after one prompt change: five rows rerun
agentkit day 13: load_evals, grade, run_eval, 60 tests; evals.csv, the first run
Start day 13: the earlier modules and agentkit as day 12 left it, 54 tests
```

Every passed and every check the same on both lines; only requests and dollars moved.

The AI check

The docstring is the prompt. The agent code that comes back runs and looks right.

1

Docstring

True when every expected number appears in the run's final answer.

→

2

Code back

Written for the course in the style of an AI answer. It runs.

→

3

Test first

A planted question with the key, a wrong answer: graded pass. The test fails.

→

4

Fix

Read only the final answer: (False, True).

Today: grade_answer reads the whole transcript, the question included: a question holding its key passes any answer.

Code that runs is not the same as code that is right.

What you can do now

- Answer keys decided first, refused if held
- grade: five checks; run_eval on every row
- Each failure read with its trace table
- One prompt change, read with git diff
- Cost and requests as columns; COUNTIFS

Week 3, Day 14 Guardrails and cost control